operating System

Analytical Problem

Code: CSA0417

Name : A. Lakshmi Devi

Reg no : 192525176

Cpu Scheduling - SJF Non-preemptive.

Given Processes:

process	Burst time (ms)
P_1	10
P_2	4
P_3	6
P_4	2

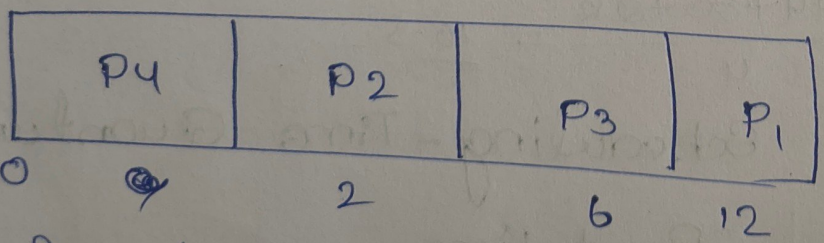
All processes arrive at time 0.

a) Calculating waiting time and turnaro-und time.

In Shortest Job first (SJF), the process with the Smallest burst time executes first.

Execution order : $P_4 \rightarrow P_2 \rightarrow P_3 \rightarrow P_1$

Gantt Chart:



formulas:-

- waiting Time (WT) = Start Time - Arrival Time.
- Turnaround Time (TAT) = Completion Time - Arrival Time.

Process	AT	BT	C.T	W.T	T.T
P_1	0	10	22	12	22
P_2	0	4	6	2	6
P_3	0	6	12	6	12
P_4	0	2	2	0	2

b) Average waiting time and turnaround ti

$$AWT = \frac{12+2+6+0}{4} = 5ms$$

$$ATAT = \frac{22+6+12+2}{4} = 10.5ms$$

c) Compare SJF with FCFS

Under FCFS, the execution order is $P_1 \rightarrow P_2 \rightarrow P_3 \rightarrow P_4$.

FCFS average waiting time:

$$\frac{0+10+14+20}{4} = 11$$

FCFS average turnaround time

$$\frac{10+14+20+22}{4} = 10.5$$

Round Robin Scheduling - Time Quantum Impact.

process	Burst time.
P_1	20
P_2	5
P_3	13
P_4	8

Quantum = 10 ms: fewer Context switches, scheduling Overhead, but processes may wait longer for their next turn.
- A larger quantum makes Round Robin behave more like FCFS.

3. Deadlock - Banker's Algorithm

a) Calculate the Need Matrix.

formula:

Need = Maximum - Allocation

process	max(A, B, C)	Allocation(A, B, C)	need (A, B, C)
P_1	(7, 5, 3)	(0, 1, 0)	(7, 4, 3)
P_2	(3, 2, 0)	(2, 0, 0)	(1, 2, 2)
P_3	(9, 0, 2)	(3, 0, 2)	(6, 0, 0)
P_4	(4, 2, 2)	(2, 1, 1)	(2, 1, 1)
P_5	(5, 3, 3)	(0, 0, 2)	(5, 3, 1)

b) Determine whether the System is in a Safe State.

$$(A, B, C) = (10, 5, 7)$$

Total allocated resources:

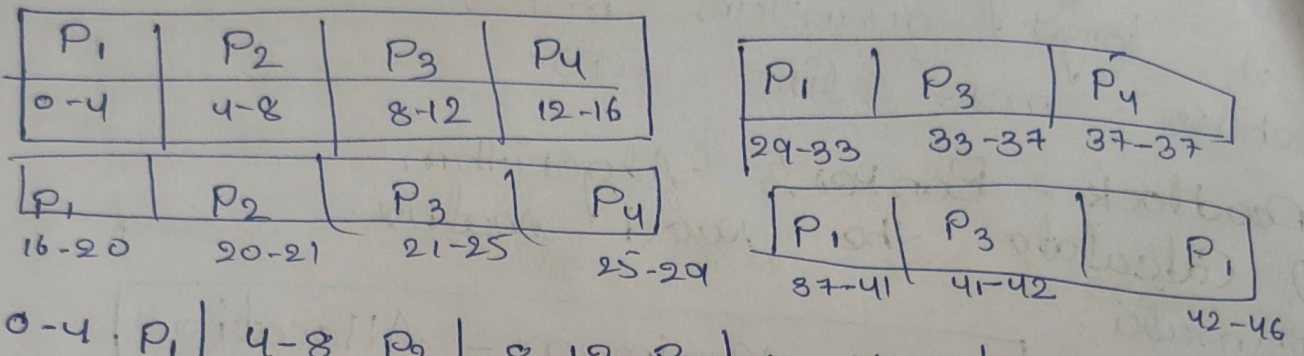
$$A = 7,\ B = 2,\ C = 5$$

Therefore, Available resources are

$$\text{Available} = (10-7, 5-2, 7-5)$$

$$\text{Available} = (3, 3, 2)$$

Time quantum = 4ms.
All processes arrive at time 0.
a) Gant chart.



0-4 P_1 | 4-8 P_2 | 8-12 P_3 | 12-16 P_4 | 16-20 P_1 | 20-21 P_2 | 21-29 P_4 | 29-33 P_1 | 33-37 P_3 | 37-41 P_1 | 41-42 P_3 | 42-46 P_1

CT

P_1 = 46ms, P_2 = 21ms, P_3 = 42ms

b) waiting time and turnaround time.

WT = CT - BT, TAT = CT

Process	B.T	C.T	W.T	T.T
P_1	20	46	26	46
P_2	5	21	16	21
P_3	13	42	29	42
P_4	8	29	21	29

Average W.T

$$AWT = \frac{26+16+29+21}{4} = 23ms$$

Average T.T = $\frac{46+21+42+29}{4} = 34.5$

c) Effect of increasing quantum to 10 ms
- Quantum = 4 ms: more frequent Context Switches and better responsiveness for interactive processes.

$R_1 \rightarrow P_1$

$R_2 \rightarrow P_2$

$R_3 \rightarrow P_4$

Request edges:

$P_1 \rightarrow R_2$

$P_2 \rightarrow R_3$

$P_3 \rightarrow R_1$

The resulting cycle is

P_1

↓ requests R_2

$R_2 \rightarrow P_2$

↓ requests R_3

$R_3 \rightarrow P_3$

↓

requests R_1

$R_1 \rightarrow P_1$

b) Determine whether the System is in deadlock.

yes, the System is in deadlock, assuming each resource has a single instance.

Reason:

1. P_1 waits for R_2, which is held by P_2.
2. P_2 waits for R_3, which is held by P_3.
3. P_3 waits for R_1, which is held by P_1.
4. All three processes wait indefinitely for resources held by one another.

Step	Process	Need	Available before execution	Available after Completion
1	P_2	(1,2,2)	(3,3,2)	(5,3,2)
2	P_4	(2,1,1)	(5,3,2)	(7,4,3)
3	P_5	(5,3,1)	(7,4,3)	(7,4,5)
4	P_1	(7,4,3)	(7,4,5)	(7,5,5)
5	P_3	(0,0,0)	(7,5,5)	(10,5,7)

c) Safe Sequence.

$P_2 \rightarrow P_4 \rightarrow P_5 \rightarrow P_1 \rightarrow P_3$

4) Deadlock Detection - Resource Allocation Graph.

Given:

- P_1 holds R_1 and waits for R_2.
- P_2 holds R_2 and waits for R_3.
- P_3 holds R_3 and waits for R_1

a) Draw or describe the Resource Allocation Graph.

In a Resource Allocation Graph (RAG).

- A process is represented by a cycle.
- A resource is represented by a circle.
- An allocation edge points from a resource to a process.
- A request edge points from a process to a resource.

Cpu Utilization - Multiprogramming level Analysis.

Given:

Each process spends 20% of its time using the cpu.

Probability of waiting for I/O: $P = 0.8$

Initial degree of Multiprogramming: $n = 4$

$$U = 1 - P^n$$

a) Calculate Cpu utilization for 4 processes.

Substitute $P = 0.8$ and $n = 4$:

$$U = 1 - (0.8)^4$$

$$U = 1 - 0.4096 = 0.5904$$

Therefore,

$$U = 59.04\%$$

b) Calculate Cpu utilization for 6 process when the degree of multiprogramming increases to 6:

$$U = 1 - (0.8)^6$$

$$U = 1 - 0.262144$$

$$U = 73.785\% \approx 73.79\%$$

c) Interpret the results:

Degree of Multiprogramming

Cpu utilization.

4 processes	59.04%.
6 processes	73.79%.
Improvement	14.75 percentage Point.